

Product Requirements Document

Product goal

Allow a parent to book and mock-pay for a child's trial class while guaranteeing an accurate confirmed roster under duplicate requests, payment failure, retries, and simultaneous attempts for the last seat.

Actors

- Parent: selects one of their children, chooses a trial class, submits a booking, records a mock payment result, and sees status.
- Admin or teacher: views the confirmed roster for a trial class.
- Payment simulator: produces a successful or failed payment result and may retry the same result.

User journeys

Successful booking

1. Parent selects a child.
2. Parent sees trial classes and indicative availability.
3. Parent creates a booking in `pending_payment`.
4. Payment simulator records success.
5. Backend atomically claims capacity and confirms the booking.
6. Parent sees `confirmed`; admin sees the child in the roster.

Failed payment

1. Parent creates a pending booking.
2. Payment simulator records failure.
3. Booking becomes `payment_failed`.
4. Child does not appear in the confirmed roster.

Last-seat loss

1. Two different students hold pending bookings when one seat remains.
2. Both receive payment success.
3. Confirmation requests contend on the same class.
4. One becomes `confirmed`; the other becomes `capacity_unavailable`.
5. The roster remains at four students.

Duplicate attempt

1. Parent attempts another active booking for the same child and class.
2. Backend returns the existing booking or a stable conflict response.
3. No second active booking or duplicate roster entry is created.

Functional requirements

ID	Requirement	Priority
FR-01	List a parent's children	Must
FR-02	List trial classes with capacity and indicative confirmed count	Must
FR-03	Create a pending trial booking	Must
FR-04	Record mock payment success or failure	Must
FR-05	Show current booking status	Must
FR-06	Show a confirmed-only class roster	Must
FR-07	Reject or reuse a duplicate active booking for the same student/class	Must
FR-08	Prevent confirmed occupancy from exceeding class capacity	Must
FR-09	Resolve simultaneous final-seat confirmation so at most one succeeds	Must
FR-10	Safely replay booking and payment commands	Should
FR-11	Explain capacity loss after successful payment	Should

Non-functional requirements

- Correctness: invariants remain true under concurrent requests.
- Auditability: booking and payment outcomes can be inspected independently.
- Reproducibility: setup, seed, and test steps are deterministic.
- Simplicity: a reviewer can run the project and understand the core path quickly.
- Traceability: each must-have acceptance criterion maps to a technical decision, implementation task, and verification case.
- Scope control: infrastructure is introduced only when it supports a stated requirement.
- Accessibility: minimal UI uses labels, keyboard-operable controls, and visible statuses.
- Security boundary: trust decisions occur server-side; demo auth shortcuts are documented.

Acceptance criteria

AC-01 Available class

Given a class has fewer than four confirmed bookings, when a student with no active booking records a successful payment, then the booking becomes confirmed and appears exactly once in the roster.

AC-02 Payment failure

Given a pending booking, when payment fails, then its status becomes `payment_failed` and it never appears in the confirmed roster.

AC-03 Duplicate booking

Given a student already has a pending or confirmed booking for a class, when another active booking is requested, then no second active booking is created.

AC-04 Full class

Given a class already has four confirmed students, when another booking receives payment success, then it does not become confirmed and the roster remains at four.

AC-05 Last-seat race

Given a class has exactly three confirmed students and two different pending bookings, when both are confirmed concurrently, then exactly one becomes `confirmed`, the other becomes `capacity_unavailable`, and the class has exactly four confirmed students.

AC-06 Retry safety

Given the same payment-result command is delivered more than once with the same idempotency key, when retries are processed, then only one payment attempt effect occurs and the booking outcome remains stable.

Status vocabulary

Status	Meaning	Occupies seat?	Terminal for demo?
<code>pending_payment</code>	Booking exists but has no successful payment result	No	No
<code>confirmed</code>	Payment succeeded and capacity was atomically acquired	Yes	Yes
<code>payment_failed</code>	Payment attempt failed	No	Yes
<code>capacity_unavailable</code>	Payment succeeded but no seat remained	No	Yes
<code>cancelled</code>	Booking was cancelled	No	Yes

Assumptions

- Class capacity is stored per class and seeded as 4.
- Availability shown before payment is advisory and may change.
- Only confirmed bookings consume capacity and appear in the roster.
- Synthetic data contains no sensitive information.
- All timestamps are stored in UTC.
- The demo uses a trusted seeded identity; production role and ownership checks are out of scope.
- The take-home is a single deployable application with one authoritative PostgreSQL database.
- A distributed Saga, Kafka, and full GitFlow are not product requirements. They are considered production/workflow alternatives and are deliberately deferred.